

 Member-only story

Docker Management Is Evolving These 8 Tools Made My Life Easier

It's 2025. Stop managing Docker like you're still debugging in Bash loops.



<devtips/>

Follow

8 min read · Apr 25, 2025



310



5



YOU'RE DOING DOCKER WRONG—8 TOOLS THAT CHANGED HOW I WORK FOREVER

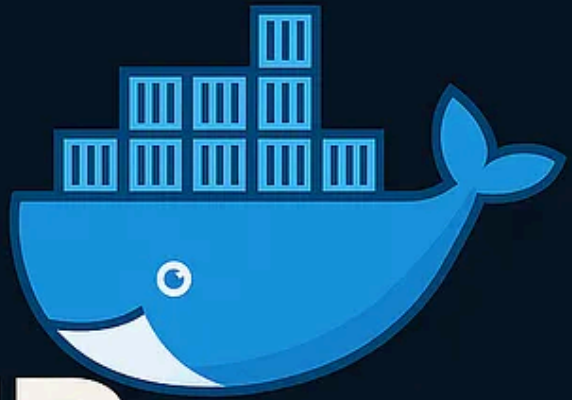


Image created using DALL-E

Introduction

If you've ever spent your Saturday night fixing a broken `docker-compose.yml` stack instead of gaming, this one's for you.

Run.it.bare — YouTube channel

Back in 2020, managing Docker felt like performing black magic. You'd build images that weighed more than your Steam library, containers crashed mid-demo, and every CI pipeline was a fragile house of cards waiting for one misplaced `ENV` line to crumble.

But times have changed.

It's 2025. The DevOps landscape around Docker has matured faster than a speedrun leaderboard. We now have battle-tested tools that solve the problems we used to just "cope with" bloated images, flaky local dev loops, zero observability, and container graveyards on our systems.

This article is not a "10 tips to optimize Docker" fluff piece. It's a real dev's survival kit tools I use daily that *actually* make container management feel smooth, even fun. Some are sleek GUIs, others are CLI power tools. A few

you probably haven't heard of. All of them changed the way I work with Docker.

Ready to unchain yourself from the old ways? Let's dive into the toolbelt.

Section 2: Why 2020 Docker Workflows Suck in 2025

Ah yes, the good ol' days of Docker in 2020:

- Running `docker ps` to check what's broken (spoiler: everything)
- Manually restarting containers like it's your side gig
- Pushing 1.2GB images to production with fingers crossed
- Forgetting to rebuild after a change and wondering why it's not working
- Managing updates by... not managing them at all

Look — Docker was (and still is) a solid tool. But the **ecosystem around it was immature**. Most of us duct-taped workflows using a mix of shell scripts, YAML sorcery, and frustration.

Here's what made those workflows painful:

1. Slow, Bloated Image Builds

You'd layer package managers, install tools you didn't need, and ship images with more junk than a 90s PC. Multi-stage builds existed, but no one used them right.

2. Zero Automation

If you weren't running Jenkins or GitHub Actions *just right*, updating a container meant SSH'ing into a server and pulling new images manually. Yes, like it's 2013.

3. No Visibility

Docker logs are useful — until they aren't. Figuring out *why* something crashed was like hunting a bug in Narnia. Want metrics? Good luck setting up Prometheus just for a dev container.

4. Painful Dev Loops

Rebuild. Restart. Realize you forgot to `COPY . .` in your Dockerfile. Rebuild again.

That's 20 minutes you're not getting back. Multiply that by 5 containers and now it's a *day*.

5. No Auto Updates, No Recovery

One outdated container turns into two, then ten. You forget what's running. Something breaks during deploy — and your only rollback plan is Ctrl+C and prayer.

We tolerated all this because we had no choice. But now? There are better tools. Smoother. Smarter. And actually fun to use.

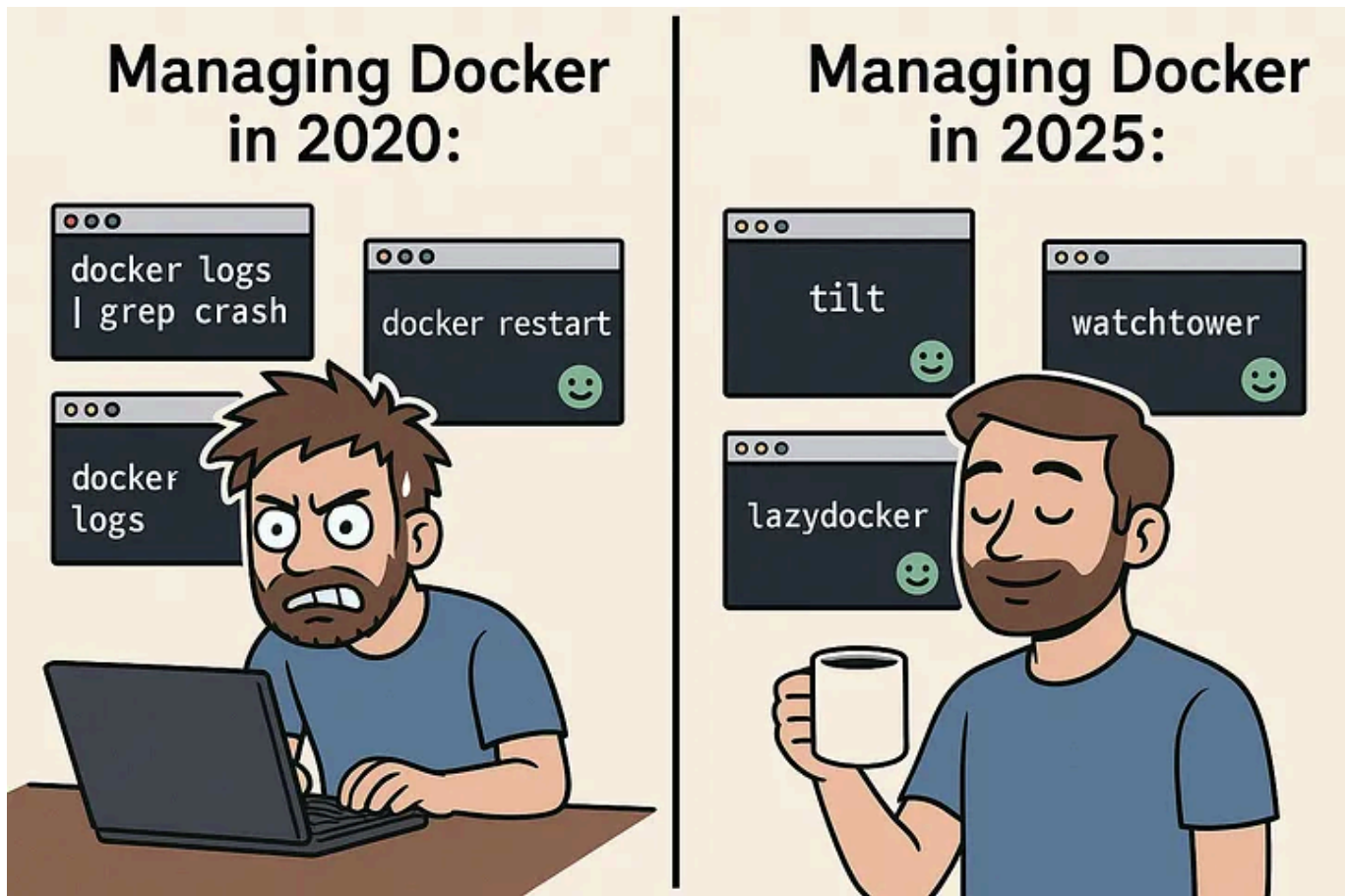


Image created using DALL-E

Section 3: The 8 Tools That Actually Made Docker Fun Again

These aren't buzzwordy "alternatives" — these are legit, practical tools that made me stop dreading container work. I'll walk you through each one, what problem it solves, and when it shines.

[Open in app](#)

≡ Medium

🔍 Search

✍ Write



Dagger lets you build CI/CD pipelines using containers as the engine — it treats infrastructure as code with a real programming model (using Go or Python). It's like Docker meets Bazel, but way friendlier.

How it works:

You define “pipelines” in code that run inside containers. You can run them locally *exactly* as they would in CI. Say goodbye to “it works on CI but not on my machine” drama.

When to use it:

- Complex CI/CD that involves multi-container builds
- Testing & deployment workflows with branching logic
- You want container reproducibility from dev to prod

When not to use it:

- Super-simple pipelines (you’ll be overengineering)
- Teams with zero devops background — there’s a learning curve

Real-world example:

We replaced 700 lines of GitHub Actions YAML with a Dagger pipeline that built, tested, pushed, and deployed 3 services — all in one composable script. Bonus: It ran the same locally.

2. OrbStack — The Fastest Docker for Mac, Period

What it is:

OrbStack is a lightning-fast Docker + Linux VM for macOS that feels like WSL for Mac — lightweight, instant startup, and *no more Docker Desktop drama*.

How it works:

It runs containers and Linux distros natively with macOS integration. Think of it as Docker Desktop without the slowness or bloat.

When to use it:

- You're a Mac dev tired of Docker Desktop
- Need fast local dev environments
- Want better resource usage and networking

When not to use it:

- You're on Linux or Windows (it's Mac-only)
- Your team relies on Docker Desktop features (like GUI settings)

Real-world example:

I replaced Docker Desktop with OrbStack. My container startup times dropped from 10 seconds to 1.5. And no more 3GB RAM usage just to keep it running.

3. Tilt — Instant Feedback for Docker-Based Dev**What it is:**

Tilt is like hot reload for your containers. It watches your code, rebuilds containers (or images), and redeploys your app automatically.

How it works:

You write a `Tiltfile` that tells Tilt how to run and reload your services. It supports Docker and Kubernetes, with UI + logs in real time.

When to use it:

- Microservices with tons of moving parts
- Kubernetes-based local dev
- You want to cut rebuild → run → debug loops

When not to use it:

- Single-container projects
- If you hate learning new syntax (Tiltfile is new-ish)

Real-world example:

Tilt helped our team of 4 devs reduce rebuild time from 2 minutes to ~10 seconds — each time someone made a code change. Massive productivity win.

4. Croc — Securely Share Docker Contexts or Files

What it is:

Croc lets you send files securely and easily between devices via CLI — like `scp`, but way cooler and safer.

How it works:

It sets up an encrypted tunnel using a relay, so you can share files (like Docker contexts, `.tar` images, `.env` files) with a single command:

```
croc send docker-context.tar.gz
```

When to use it:

- Pair programming or remote debugging
- Sharing build artifacts between devs
- You need quick secure handoffs (without Dropbox or email)

When not to use it:

- Cloud-first setups with shared volumes
- If you've got CI/CD doing all artifact management

Real-world example:

We use Croc during code pairing sessions to transfer built Docker contexts fast, encrypted, and no setup needed.

5. Podman — The Rootless Docker Alternative

What it is:

Podman is a drop-in Docker CLI replacement that runs containers rootlessly — no daemon, no socket binding, just containers.

How it works:

You run `podman` like `docker`, but it works without a daemon. You can also run `systemd` inside containers (super cool for services). It even supports pods like Kubernetes.

When to use it:

- You want secure containers in dev and prod
- You hate the Docker daemon crashing
- You're into automation or rootless deployments

When not to use it:

- You're deeply tied into Docker-only tooling
- GUI-based Docker management is a must

Real-world example:

We use Podman for CI containers on our Linux runner — no daemon, no root, less risk.

6. Lazydocker — A TUI That Makes Docker Human

What it is:

Lazydocker is a terminal UI for managing Docker containers, volumes, images, logs, and more — visually, from your CLI.

How it works:

Just run `lazydocker` and you get a curses-style interface. Scroll through containers, inspect logs, restart services, see CPU/mem usage.

When to use it:

- You're tired of typing `docker logs` 100 times a day
- You want quick insights into container health
- Local dev/debug sessions

When not to use it:

- Headless servers with no terminal access
- You need full-blown orchestration

Real-world example:

I use Lazydocker like a mission control dashboard for my dev stack — one pane, all the info I need.

7. Dive See Why Your Docker Image Is So Huge**What it is:**

Dive lets you inspect and analyze each layer of a Docker image to see what's bloating your builds.

How it works:

You run `dive your-image` and it shows you what each layer added — files,

sizes, commands. You get a score too.

When to use it:

- You want smaller, faster Docker images
- Debugging CI time
- You're optimizing for Fargate, Lambda, etc.

When not to use it:

- You don't care about image size (aka you're Google)
- One-off throwaway containers

Real-world example:

Cut a 700MB Node.js image down to 250MB using Dive — deleted unused global installs and layered smart.

8. Watchtower Keep Containers Updated. Automatically.

What it is:

Watchtower watches your running containers and updates them when a new image version is available.

How it works:

It checks your Docker registry, pulls the latest image, gracefully stops the old container, and spins up the new one — all automatically.

When to use it:

- Self-hosted apps that need to stay updated
- Teams without full CI/CD coverage
- Homelabs or one-man infra setups

When not to use it:

- Critical infrastructure that needs manual deploy control
- Canary/staged deploy setups

Real-world example:

Watchtower auto-updates our dev tools like Portainer, Postgres admin, and Redis UI — we never touch them again after setup.

Section 4: Tool Comparison Table

Because developers love charts almost as much as they hate writing tests.

Tool	Best Use Case	Works With Docker?	Local Dev	CI/CD	Observability	Rootless	Auto Updates
Dagger	CI/CD pipelines using containers	✓ Native support	✗	✓	♦ Basic logs	✓	✗
OrbStack	Fast Docker and Linux dev on macOS	✓ Docker backend	✓	✗	♦ Minimal	✓	✗
Tilt	Live dev with smart reloads	✓ Docker & K8s	✓	✗	✓ Real-time logs	✓	✗
Croc	Securely share Docker contexts or files	⊖ Works alongside	✓	✗	✗	✓	✗
Podman	Secure, daemonless container runtime	✓ Docker CLI	✓	✓	♦ CLI-level only	✓	✗
Lazydocker	Terminal UI for managing containers	✓ Docker & Podman	✓	✗	✓ Live usage views	✓	✗
Dive	Image inspection and optimization	✓ Docker images	✓	✓	✓ Layer-level view	✓	✗
Watchtower	Auto-updating running containers	✓ Watches images	✓	✗	✗	✓	✓ ✓

Table created using Gemini

♦ = Limited or basic support

✓ = Yes

✗ = Nope

⊖ = Not directly but plays nice

Section 5: Conclusion — Don't Just Use Docker. Master It.

Docker's not dead. It just... evolved. And managing it like it's still 2020 is a waste of your time and sanity.

These 8 tools didn't just make my setup cleaner — they made my **workflow faster, my images smaller, and my deployments smoother**. They helped me debug less, automate more, and actually enjoy working with containers again.

So no, you don't need to abandon Docker. You just need to **upgrade the tools around it**. Use what fits your workflow. Don't try all eight at once — pick one or two that solve your biggest pain right now. You'll feel the difference instantly.

And hey, if you're still restarting containers manually in 2025... just know there's a better life out there.

Helpful Dev Resources (Bookmark These)

- [Dagger Docs](#) — CI/CD done right, with containers
- [OrbStack](#) — The Mac-native Docker dev environment you wish existed sooner
- [Tilt](#) — Live reload and dev automation for container stacks
- [Croc GitHub](#) — Secure file and Docker context sharing
- [Podman](#) — Run containers securely, rootlessly
- [Lazydocker GitHub](#) — TUI to manage containers without rage
- [Dive GitHub](#) — Cut image size and see what's really inside

- **Watchtower** — Let your containers update themselves
- **Docker's Blog** — Still useful, still evolving

Enjoyed this story?

If you liked it, please leave a comment sharing what topic you'd like to see next!

Feel free to **like**, **share** it with your friends, and **subscribe** to get updates when new posts go live. 🚀

Docker +

Technology +

Programming ✓

Cloud Computing +

Database ✓

**Written by <devtips/>**

13.3K followers · 93 following

Follow

Dev tips for folks who break things on purpose. Cloud, AI, and DevOps workflows that work. Daily stories, scars, and lessons. videos: youtube.com/@Runitbare

